

Informe del módulo de indexación

CSDI RAG Engine

1 Módulo de indexación

El módulo de indexación es el componente encargado de transformar los documentos adquiridos por el sistema en estructuras de datos eficientes para la recuperación de información. En el proyecto, este módulo trabaja principalmente sobre fragmentos textuales derivados de páginas de documentación técnica, como la documentación oficial de Python y la documentación de JavaScript en MDN. Su salida principal es un índice invertido persistente que permite localizar rápidamente los documentos que contienen los términos de una consulta y calcular una puntuación de relevancia mediante BM25.

La indexación no se implementa como una simple tabla de documentos completos. El sistema mantiene información estadística necesaria para recuperación léxica: vocabulario, listas de ocurrencias, frecuencia de términos, longitud de documentos y estadísticas globales del corpus. Estas estructuras permiten que el recuperador no tenga que recorrer todos los documentos en cada consulta, sino únicamente las listas de documentos asociadas a los términos buscados.

Selección de BM25 como modelo básico de recuperación. Para la recuperación básica del sistema se seleccionó BM25, una función de ranking probabilística ampliamente utilizada en sistemas de recuperación textual. La elección responde a tres razones principales. Primero, BM25 ofrece una base sólida y explicable para ordenar documentos según coincidencia léxica, frecuencia de términos e importancia global de cada término en el corpus. Segundo, se adapta bien a corpus de documentación técnica, donde los usuarios formulan consultas con nombres de funciones, clases, errores, conceptos del lenguaje y palabras clave que suelen aparecer de forma explícita en los documentos relevantes. Tercero, su implementación puede apoyarse directamente en un índice invertido, estructura eficiente y apropiada para el módulo de indexación requerido en el proyecto.

El dominio temático del sistema es tecnología y software, específicamente documentación técnica. En este contexto, la coincidencia lexical tiene un valor alto. Si un usuario consulta `python list comprehension`, `asyncio task`, `javascript promise` o `array map`, los términos de la consulta no son meramente descriptivos: suelen coincidir con identificadores, conceptos o secciones concretas de la documentación. BM25 aprovecha esa propiedad porque favorece documentos donde esos términos aparecen con frecuencia suficiente, pero evita que un documento largo sea favorecido injustamente solo por contener más texto.

Además, BM25 funciona como una base robusta dentro de una arquitectura híbrida. Aunque el sistema también incorpora recuperación vectorial y RAG, BM25 conserva ventajas importantes: recupera con precisión nombres exactos de APIs, operadores, mensajes de error y términos técnicos raros. Por esa razón, su papel como modelo básico no es únicamente cumplir un requisito inicial, sino aportar una señal de relevancia complementaria y verificable dentro del recuperador general.

Fundamento técnico de BM25. BM25 asigna a cada documento una puntuación con respecto a una consulta. Si la consulta contiene varios términos, la puntuación final se calcula como la suma de las contribuciones de cada término presente en el documento. En el sistema se utiliza la forma:

$$\text{score}(D, Q) = \sum_{q_i \in Q} \text{IDF}(q_i) \cdot \frac{tf(q_i, D)(k_1 + 1)}{tf(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}}\right)}$$

donde $tf(q_i, D)$ es la frecuencia del término q_i en el documento D , $|D|$ es la longitud del documento, avgdl es la longitud promedio de los documentos del corpus, k_1 controla la saturación de la frecuencia de término y b controla la normalización por longitud. En la implementación del proyecto, los valores por defecto son $k_1 = 1.5$ y $b = 0.75$, valores usuales para recuperación textual general.

El componente de frecuencia de término no crece linealmente. Esto es importante porque una palabra repetida muchas veces no debe aumentar indefinidamente la relevancia del documento. Por ejemplo, si un fragmento menciona **decorator** dos veces y otro lo menciona veinte veces, el segundo puede ser más relevante, pero no diez veces más relevante. BM25 aplica una saturación: las primeras ocurrencias aportan más información, mientras que las repeticiones posteriores tienen un efecto marginal menor.

La normalización por longitud evita favorecer documentos extensos. En documentación técnica, una página de referencia muy larga puede contener una gran cantidad de términos solo por su tamaño. BM25 compensa ese efecto mediante el factor $\frac{|D|}{\text{avgdl}}$. Si un documento es más largo que el promedio, necesita una frecuencia de término proporcionalmente mayor para obtener la misma evidencia de relevancia que un fragmento corto y específico. Esto es adecuado para el sistema porque los documentos se dividen en chunks: un fragmento corto que explica directamente **list comprehension** debe competir favorablemente contra una página extensa que menciona el término de manera secundaria.

La importancia global del término se calcula mediante IDF:

$$\text{IDF}(q_i) = \log \left(\frac{N - df(q_i) + 0.5}{df(q_i) + 0.5} + 1 \right)$$

donde N es el número total de documentos indexados y $df(q_i)$ es la cantidad de documentos que contienen el término. Esta parte de la fórmula penaliza términos frecuentes en el corpus y favorece términos discriminativos. En un corpus de documentación de Python, palabras como **function**, **object** o **return** pueden aparecer en muchas páginas, mientras que términos como **dataclass**, **asyncio**, **itertools** o **decorator** son más informativos para localizar secciones específicas.

Ejemplo de interpretación de BM25. Supóngase una consulta **python generator expression**. Después de la normalización, el sistema trabaja con términos derivados como **python**, **generat** y **express**. Para cada término se consulta el índice invertido. Si **generat** aparece en tres documentos y **python** aparece en casi todos, entonces **generat** tendrá una IDF mayor y aportará más al ranking. Un documento corto que explique directamente generadores y expresiones generadoras obtendrá una puntuación alta porque contiene términos discriminativos con buena frecuencia y una longitud cercana o menor al promedio. En cambio, una página general que mencione **python** muchas veces, pero no explique generadores, recibirá una puntuación menor porque el término común aporta poca evidencia de relevancia.

Este comportamiento es deseable en el dominio escogido. Las consultas técnicas suelen contener uno o dos términos realmente decisivos. El modelo debe distinguir entre documentos que solo comparten vocabulario general del dominio y documentos que contienen el concepto técnico preciso que el usuario busca.

Diseño general del flujo de indexación. El flujo de indexación comienza después de la adquisición y segmentación del contenido. Cada documento fuente se divide en fragmentos indexables o

chunks, los cuales conservan metadatos como identificador, fuente, URL, título y ruta de navegación. El servicio de ingesta recibe estos fragmentos, elimina duplicados a partir del identificador del chunk, persiste sus metadatos y luego envía el texto al indexador.

El proceso de indexación léxica sigue una secuencia definida: tokenización del texto, conteo de frecuencias, incorporación al estado activo del índice, construcción de segmentos persistentes, posible mezcla de segmentos y recarga del recuperador. Esta separación permite que la ingesta no dependa de los detalles internos de BM25 y que el recuperador pueda cargar una vista consistente del índice cuando necesita responder consultas.

Normalización y tokenización. Antes de construir el índice, el texto de cada chunk se normaliza mediante una función compartida por la indexación y la búsqueda. El texto se convierte a minúsculas, se extraen secuencias alfanuméricas, se eliminan palabras vacías en inglés y se aplica stemming con Snowball. Esta consistencia es fundamental: si el documento se indexa con una transformación y la consulta se procesa con otra, términos conceptualmente iguales podrían no coincidir.

Por ejemplo, expresiones como `decorators`, `decorated` y `decorator` pueden reducirse a una raíz común. Así, una consulta formulada con una variante morfológica puede recuperar documentos que usan otra. En documentación técnica esto resulta útil porque las páginas pueden alternar entre el nombre del concepto, su uso verbal y descripciones derivadas. También se eliminan términos de bajo valor discriminativo como artículos, preposiciones y conectores, lo que reduce ruido en el índice y en el cálculo de ranking.

Construcción del índice invertido. La estructura central es el índice invertido. En lugar de guardar únicamente, para cada documento, la lista de términos que contiene, el índice guarda, para cada término, la lista de documentos en los que aparece. Cada entrada de esa lista, conocida como *posting*, contiene al menos el identificador del documento y la frecuencia del término dentro de ese documento.

Como ejemplo simplificado, supóngase que se indexan tres chunks:

```
D1: python list comprehension list
D2: python generator yield
D3: javascript array map
```

Después del conteo de términos, una parte del índice invertido podría representarse así:

```
python    → [(D1, 1), (D2, 1)]
list      → [(D1, 2)]
generator → [(D2, 1)]
javascript → [(D3, 1)]
```

El diccionario del índice almacena cada término y su frecuencia documental, es decir, cuántos documentos contienen el término. Las listas de postings almacenan la evidencia necesaria para calcular $tf(q_i, D)$. En paralelo, el módulo mantiene un mapa de longitudes documentales, por ejemplo $|D_1| = 4$, $|D_2| = 3$, $|D_3| = 3$, y estadísticas globales como el número total de documentos y la longitud promedio. Sin estas estadísticas, no sería posible calcular BM25 de forma correcta.

En la implementación, el estado activo del índice mantiene tres estructuras principales: diccionario y postings dentro del índice invertido, longitudes por documento y estadísticas del corpus. Cuando se registra un documento, primero se calcula su tabla de frecuencias locales. Luego, por cada término, se añade un posting con el identificador del chunk y su frecuencia. Por último, se actualiza la longitud del documento y los acumulados del corpus.

Indexación segmentada. El sistema no reescribe un único índice completo cada vez que entra un documento. En su lugar, utiliza una estrategia de indexación segmentada. Los chunks nuevos se acumulan temporalmente en memoria dentro de un estado activo. Cuando se alcanza el tamaño de buffer configurado, cuando transcurre el intervalo máximo de escritura o cuando finaliza una ingesta, el estado activo se materializa como un segmento inmutable.

Un segmento contiene una instantánea del índice invertido correspondiente a un conjunto de documentos: diccionario, postings, longitudes documentales y estadísticas. Esta decisión tiene ventajas prácticas. Primero, reduce el costo de escritura porque se agrupan múltiples documentos en una operación lógica. Segundo, evita modificar constantemente estructuras ya persistidas. Tercero, permite reconstruir y mezclar segmentos de manera controlada sin perder consistencia.

La configuración por defecto del módulo usa un tamaño de buffer de 10000 documentos, un máximo de 5 segmentos activos antes de iniciar mezclas y un intervalo de flush de 30 segundos. Estos valores pueden ajustarse desde variables de entorno, lo cual permite adaptar el comportamiento del indexador al tamaño del corpus y a las condiciones de ejecución.

Persistencia del índice. Cada segmento se persiste en PostgreSQL mediante un conjunto de tablas especializadas. La tabla de segmentos almacena el identificador del segmento, fecha de creación, cantidad de documentos, cantidad total de términos y longitud promedio. La tabla de términos almacena el vocabulario de cada segmento y la frecuencia documental de cada término. La tabla de postings almacena, para cada término, los documentos en los que aparece y su frecuencia local. La tabla de longitudes documentales conserva la longitud de cada chunk.

Esta representación relacional permite reconstruir el índice completo después de reiniciar la aplicación. El recuperador BM25 carga todos los segmentos desde la base de datos, reconstruye las listas de postings en memoria y calcula las puntuaciones de las consultas sobre esa estructura. De esta forma, la persistencia no es solo almacenamiento histórico: es la base que permite que el sistema sea reproducible y no dependa de estructuras volátiles.

Mantenimiento del índice: flush y merge. El mantenimiento del índice se realiza mediante dos operaciones principales: *flush* y *merge*. El flush toma los documentos acumulados en memoria y los convierte en un segmento persistente. Puede ocurrir automáticamente cuando el buffer alcanza su tamaño máximo, de forma periódica por tiempo o de manera forzada al finalizar una ingesta. Esto garantiza que los documentos adquiridos no permanezcan indefinidamente en memoria y puedan ser recuperados por el sistema.

El merge controla el crecimiento del número de segmentos. Si se acumulan demasiados segmentos, el sistema selecciona los más antiguos y los combina en uno nuevo. Durante esa mezcla se unen las listas de postings, se recalculan las frecuencias documentales, se consolidan las longitudes de documentos y se actualizan las estadísticas globales del segmento resultante. Después, el nuevo segmento reemplaza de forma atómica a los segmentos originales. Esta operación mantiene el índice manejable y evita que la recuperación tenga que consultar una cantidad excesiva de segmentos pequeños.

Integración con la ingesta de chunks. La indexación está integrada con el servicio de ingesta de fragmentos. Antes de indexar, el sistema consulta qué identificadores de chunks ya existen y descarta los repetidos. Esta deduplicación es relevante porque un mismo documento fuente puede ser adquirido más de una vez, especialmente durante pruebas o actualizaciones del corpus. Si el mismo chunk se indexara varias veces, BM25 sobreestimaría su relevancia porque sus postings y longitudes aparecerían duplicados.

Después de filtrar los chunks nuevos, el servicio guarda sus metadatos y ejecuta la indexación. Para la parte léxica, tokeniza el texto y llama al constructor del índice. Para la parte vectorial, construye un texto enriquecido con título, breadcrumb y contenido, y lo envía al indexador vectorial. Aunque este informe se concentra en el índice invertido, esta integración es importante porque el sistema completo utiliza recuperación híbrida: BM25 aporta precisión léxica y la base vectorial aporta similitud semántica.

Integración con el recuperador BM25. El recuperador BM25 consume directamente el resultado del módulo de indexación. Al iniciar, carga desde la base de datos todos los postings, el diccionario, las longitudes documentales y las estadísticas globales. Luego, cuando recibe una consulta, aplica la misma tokenización usada durante la indexación, obtiene las listas de postings de cada término y acumula las contribuciones BM25 para cada documento candidato.

Este diseño evita recorrer documentos irrelevantes. Si la consulta contiene `asyncio task`, el recuperador solo necesita examinar documentos presentes en las listas de postings de `asyncio` y `task`. Los documentos que no contienen ninguno de esos términos no se evalúan en BM25, lo que reduce el costo de la búsqueda y permite ordenar resultados de forma eficiente. Finalmente, los documentos candidatos se ordenan por puntuación descendente y se devuelven los primeros k resultados solicitados.

API del módulo de indexación. El módulo expone una API propia dentro del servidor. El endpoint `POST /api/v1/indexing/index` permite incorporar un documento ya tokenizado al índice activo. Aunque en el flujo normal la indexación ocurre desde la ingesta de chunks, este endpoint facilita pruebas y operaciones controladas sobre el índice. Además, el endpoint `POST /api/v1/indexing/merge` permite ejecutar manualmente la mezcla de segmentos.

La existencia de estos endpoints muestra que la indexación no es una rutina interna aislada, sino un módulo operativo dentro de la arquitectura del sistema. Puede recibir documentos, reportar si se produjo un flush, devolver el identificador del segmento creado y ejecutar mantenimiento bajo demanda.

Pruebas y validación funcional. El repositorio incluye pruebas específicas para validar el comportamiento del módulo. Las pruebas del flujo BM25 verifican que se puedan añadir documentos, que el flush persista segmentos, que el tamaño del buffer dispare escrituras automáticas, que no se acepten documentos duplicados dentro del mismo segmento activo y que el recuperador encuentre documentos previamente indexados. También se comprueba que los resultados respeten el límite `top_k`, que las puntuaciones sean positivas para coincidencias válidas y que un documento más relevante aparezca por encima de uno irrelevante.

Adicionalmente, existen pruebas para la política de mezcla de segmentos y para el cálculo de BM25. Estas validan casos límite como frecuencia de término cero, frecuencia documental cero, corpus vacío, normalización por longitud y crecimiento de la puntuación ante mayor frecuencia del término. Con ello se cubren tanto aspectos estructurales del índice como propiedades del modelo de ranking.

Justificación técnica de la solución. La solución implementada cumple el objetivo del módulo de indexación porque construye y mantiene estructuras especializadas para recuperación eficiente. El índice invertido reduce el espacio de búsqueda de cada consulta, BM25 proporciona una función de ranking clara y adecuada para texto técnico, y la persistencia segmentada permite mantener el índice entre ejecuciones sin reindexar todo el corpus desde cero.

La separación entre adquisición, segmentación, indexación y recuperación también mejora la mantenibilidad del sistema. El módulo de adquisición se ocupa de obtener documentos limpios; el servicio de ingesta prepara chunks y evita duplicados; el indexador transforma texto en estructuras recuperables; y el recuperador usa esas estructuras para responder consultas. Esta división permite evolucionar cada componente sin mezclar responsabilidades y ofrece una base sólida para la integración posterior con recuperación vectorial, búsqueda híbrida y RAG.